day 22

hitherto

I discovered a problem with our secant method and have a fix for it below. This is embarrassing but it is important to realize that it can be hard to figure out a bug with code when the code almost works. There was nothing telling me it was broken and the answers it gave were frequently correct, but the problem was deeply buried and finally raised its head the other day (day 20) when we were working on solving a fairly simple equation. So go check the updated notes from day 20 to see what happened.

herein

We did this in class in excel but then bonked when things weren't working correctly. So here is a correct way to do it for next time.

We were learning Euler's (pronounced Oilah's) method in class and it is very simple to put into play with Excel to start off with. We will start off with first order differential equations. So for example we came up with this equation:

$$\frac{dy}{dx} = x^2 + x - 1 \text{ where } y(0) = -1$$

This is easy enough to simply **integrate**, and that is a good thing to know! Very often in DiffEQ (that's what we call Differential Equations) simply integrating, or even guessing and checking is a good way to solve things. What we are doing in Computational Physics is learning a slightly different way to calculate the answer, but even we need something to compare our answer to. So back to it.

Using Euler's method we approximate the next place with the following bit of logic:

$$y_{n+1} = y_n + \frac{dy}{dx} \cdot dx$$

So that is exactly the logic we put into Excel.

x	y	dydx
0	initial condition	diff eq
step	=B2 + C2*(A3-A2)	diff eq
...	...	...

I don't know if that table helps but that is how we do it. And we can check this by plotting the actual solution, since this is a simple enough expression that we can use some simple math to find it.

$$\int (x^2 + x - 1)dx = \frac{1}{3}x^3 + \frac{1}{2}x^2 - x + C$$

and we can find *C* by using our initial condition $y(0) = -1. That means$C=-1$. And this works great!

And we could do something like this for every **first order** differential equation. So another one we had in class was this:

$$\frac{dy}{dx} = x^2 e^{-x} - 1 \text{where } y(0) = -1$$

and we got a solution for that which I won't even bother to check.

Another one is this one

$$\frac{dy}{dx} = -y$$

Now that one seems a little more difficult because it is not as straightforward to integrate. But it is still a *separable* differential equation, since I can separate the variables to their own sides. Like this:

$$\frac{dy}{y} = -dx$$

Now I can integrate both sides:

$$\int \frac{dy}{y} = -\int dx$$

and that means

$$\ln(y) = -x + C$$

which further simplifies to

$$y = e^{-x+C} = Ae^{-x}$$

And **THAT** is the step I bonked on in class today. But we are now able to solve for that constant `A` by doing this:

$$y(0) = -1 = A$$

So that tells us exactly what to do with our "correct" solution. Check out `day22-excel-file.xsls` for the details on how to plug this into excel and what the plots

should look like. Below, I have included how to plot these files quickly using `pandas`. This is slightly different to how we have done it in the past but it is good to learn a few differences. Just make sure that the excel file is in the same directory as the jupyter file you are in.

```
df = pd.read_excel('day22-excel-file.xlsx', sheet_name='Sheet2')

df.plot('x', 'y')
```

In [3]:
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
```

In [2]:
```
def secant_bad(f, guess, delta, tolerance = 2**-32):
    x0 = guess
    x1 = x0 + delta
    n = 0
    while abs(f(x1))>tolerance:
        x1 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
        n += 1
    return(x1, n)


def secant(f, guess, delta=1, tolerance = 2**-32):
    x0 = guess
    x1 = x0 + delta
    n = 0
    while abs(f(x1))>tolerance:
        x2 = x1 - (x1-x0)/(f(x1)-f(x0))*f(x1)
        x0 = x1
        x1 = x2
        n += 1
    return(x1, n)
```

In [8]:
```
df = pd.read_excel('day22-excel-file.xlsx', sheet_name='Sheet2')#, engine='c
df
```

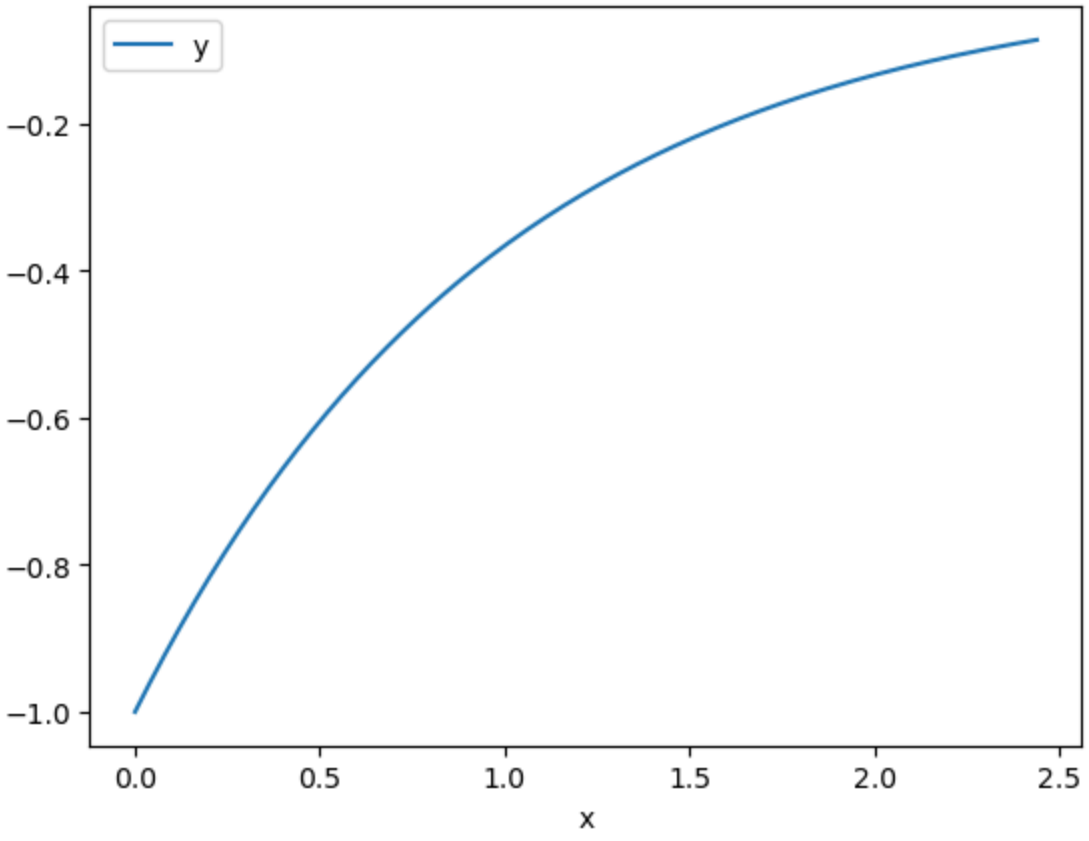
Out[8]:

	x	y	dy	f(x)	error
0	0.00	-1.000000	1.000000	-1.000000	0.000000
1	0.01	-0.990000	0.990000	-0.990050	0.005033
2	0.02	-0.980100	0.980100	-0.980199	0.010067
3	0.03	-0.970299	0.970299	-0.970446	0.015100
4	0.04	-0.960596	0.960596	-0.960789	0.020132
...	...	...	...	...	...
240	2.40	-0.089629	0.089629	-0.090718	1.200793
241	2.41	-0.088732	0.088732	-0.089815	1.205766
242	2.42	-0.087845	0.087845	-0.088922	1.210739
243	2.43	-0.086967	0.086967	-0.088037	1.215711
244	2.44	-0.086097	0.086097	-0.087161	1.220683

245 rows × 5 columns

In [9]:
```
df.plot('x', 'y')
```

Out[9]:
```
<Axes: xlabel='x'>
```



In [ ]: